

MultiROS: ROS-Based Robot Simulation Environment for Concurrent Deep Reinforcement Learning

Jayasekara Kapukotuwa^a, Brian Lee^a, Declan Devine^b and Yuansong Qiao^a

Abstract — On the journey of true autonomous robotics, applying deep reinforcement learning (DRL) techniques to solve complex robotics tasks has been a growing interest in academics and the industry. Currently, numerous simulation frameworks exist for evaluating DRL algorithms with robots, and they usually come with prebuilt tasks or provide tools to create custom environments. Among these, one of the highly sought approaches is using Robot Operating System (ROS) based DRL frameworks for simulation and deployment in the real world. The current ROS-based DRL simulation frameworks like `openai_ros` or `Gym-gazebo` provide a framework for creating environments; however, they do not support training with vectorised environments for speeding up the training process and parallel simulations for testing and evaluating meta-learning, multi-task learning and transfer learning approaches. Therefore, we present MultiROS, a 3D robotic simulation framework with a collection of prebuilt environments for deep reinforcement learning (DRL) research to overcome these limitations. This package interfaces with the Gazebo robotic simulator using ROS and provides a modular structure to create ROS-based RL environments. Unlike the others, MultiROS provides support for training with multiple environments in parallel and simultaneously accessing data from each simulation. Furthermore, since MultiROS uses the popular OpenAI Gym interface, it is compatible with most OpenAI Gym based reinforcement learning algorithms that use third-party python frameworks.

I. INTRODUCTION

Simulations have been highly sought after since the early days of robotics. The immense popularity of the simulators is due to their ability to limit the risks that expensive hardware such as robots must face during the prototyping period. As good practice, most roboticists usually test their code in a simulation before deploying it to real robots. Due to the innovations and improvements in stimulation technologies, simulators now can quite accurately approximate the real world with their advanced physics engines. Similarly, the traditional approaches for programming robots are also gradually fading away with the rapid advances in artificial intelligence and Machine Learning (ML) techniques. This growing interest in machine learning for robotics is due to the ease of access, the availability of large amounts of data, and the increased demand for flexibility requirements of Smart Manufacturing. Coupled with the current computationally faster machine learning accelerator hardware, it is now simply possible to incorporate data-hungry deep learning (DL) techniques to program robots. In the last decade, the

advances in deep learning led a flurry of roboticists and researchers to apply DL techniques to robotics. In particular, the combination of Deep Learning and Reinforcement Learning (RL) opened a pathway to solving complex decision-making problems that previously required rigorous and delicate hand-designed approaches. Using mathematically rich tabular RL algorithms to discover optimal behaviour for operating robots indeed started many years ago [1]; however, its performance took a massive leap due to the powerful capabilities of Deep Reinforcement Learning (Deep RL) [2] algorithms, which use Artificial Neural Networks (ANN).

One of the significant hurdles in training a robot with reinforcement learning is that the RL algorithms need to dynamically interact with the environment and collect data to make optimal decisions. However, testing an RL-based approach directly in a real robot may not be ideal since the exploration during the training phase may pose a significant danger to the robot platform and its surroundings. Moreover, even with taking every safety precaution, the reinforcement learning-based algorithms still require sampling a large amount of experience to find the optimal policy (exploration). However, collecting these data in real life is time-consuming and tedious since it requires the roboticist to monitor the whole training process constantly and rearrange the environment at the end of each training episode. Furthermore, the paper by Gabriel et al. [3] outlines multiple high-level challengers in applying RL to real-world systems, and robotic applications are subjected to every single one. Therefore, the motivation is to collect samples in a simulated environment which can theoretically provide an infinite amount of data to train the RL algorithms and transfer the learned policies to the real-world robots. This process is known as Sim-to-real transfer [4], which focuses on addressing the difficulties (reality-gap) with transferring from simulation to the real world. Defining approaches for tackling reality-gap and comparison with other methods go beyond the scope of this paper, and for a detailed analysis of reality-gap in robotic manipulation tasks, we redirect the readers to the specific work of Collins et al. [5].

Currently, using Robotic Operating System (ROS) [6], which is considered the standard for programming robots together with simulators such as Gazebo [7] for creating RL environments, is in great demand due to the realistic physics approximation and ease of transferring the learned policies to real robots. Furthermore, unlike other RL frameworks, ROS-based ones have the ability to easily interchange simulation with real robots both in the training and the deployment in the real world. In the recent literature, there are many successful applications for training deep RL algorithms in single-task learning environments [8]. However, the current

^a Software Research Institute, Technological University of the Shannon: Midlands Midwest, Athlone, Co Westmeath, Ireland.

^b Materials Research Institute, Technological University of the Shannon: Midlands Midwest, Athlone, Co Westmeath, Ireland.
j.kapukotuwa@research.ait.ie, Brian.Lee@tus.ie, Declan.Devine@tus.ie, ysqiao@research.ait.ie

Table 1: Comparison of Current ROS-based Robotic Reinforcement Learning Frameworks that are Compatible with OpenAI Gym

Framework	Custom Environments	Prebuilt Environments	Documentation	Modular	Real-time Compatible	Muti-task & Meta-Learning
Gym-Gazebo	✓	✓	×	×	×	×
Openai_ros	✓	✓	✓	✓	×	×
MultiROS	✓	✓	✓	✓	✓	✓

research interests in robotics are moving toward more complex RL approaches such as Meta-learning and Multi-Task learning, where robots leverage prior experience to acquire new skills more efficiently [9]. Unfortunately, the current offerings for ROS-based RL frameworks do not support training these complex RL approaches in their task suites.

Therefore, we proposed a ROS-based framework for testing and evaluating deep RL algorithms in robotic environments. The main focus of this package is to introduce a ROS-based RL structure for multi-robot/task learning research. Its architecture extends the `openai_ros` [10] and `gym-gazebo` [11] packages, which use the OpenAI Gym interface [12] with ROS. Like in OpenAI Gym, the MultiROS structure enables RL algorithms to interact with environments to sample experiences to improve its policy. Therefore, by making minor modifications, this toolkit is compatible with most third-party python frameworks that provide RL algorithms. Furthermore, since the gazebo simulator’s realistic simulation is relatively complex compared to other counterparts, it introduces vast timing overheads for sampling experience from the training environments. To overcome this issue, the MultiROS support initialising multiple parallel gazebo instances (vectorised environment) to sample experience and speed up the learning process effectively.

The MultiROS package also contains multiple prebuilt environments for users to quickly evaluate RL algorithms without creating a new environment from scratch. It includes various robots that perform the same type of tasks, and each robot’s task environment is practically the same except for the robot and the gripper they use for executing the task. The intuition behind this selection is that it allows the users to test multi-task learning algorithms to train robots for multiple tasks. Furthermore, due to the similarity of the prebuilt tasks of the robots, the users can test the transfer of the knowledge learned in one task environment to perform another using transfer learning approaches.

This paper is structured as follows. First, we look at the currently available frameworks that provide robotic simulations for RL research and describe their strengths and shortcomings. We then proceed to present the overall architecture of the MultiROS and dive deep into the detailed structure, explaining its current features. Then we briefly explain how users can utilise the framework to create custom environments or evaluate their RL algorithms with the current prebuilt environments of MultiROS. Next, we present an experiment conducted with MultiROS for improving the training speed in a vectorised environment. Finally, we

conclude by discussing this framework’s future work and limitations.

II. RELATED WORK

The OpenAI Gym toolkit is one of the pioneering platforms to develop, compare and benchmark reinforcement learning algorithms. As one of the most popular frameworks for reinforcement learning, the OpenAI Gym supports a variety of task environments spanning from classical Atari games to complex robot control. It uses the popular MuJoCo¹ physics engine to simulate robotic task environments of the OpenAI Gym. This MuJoCo simulator is a relatively lightweight software compared with Gazebo and is currently made free by the DeepMind Robotics Simulation team. In addition, DeepMind recently released its reinforcement learning framework known as `dm_control` [13] for articulated-body simulations and robot manipulation tasks. Also inspired by the OpenAI Gym, Gallouédec et al. introduced PyBullet² based Panda-gym [14] to simulate multi-goal environments. Unfortunately, these frameworks mainly focus on single-task learning settings and do not support meta-learning and multi-robot learning research in their RL environments. To address these challenges, the recent frameworks such as gym-extensions [15], myGym [16], Robosuite [17], SURREAL [18], RLbench [19] and Meta-World [20] presented a collection of robotic task environments for evaluating meta-learning, multi-task transfer and multi-scale control algorithms. These frameworks are perfect for quickly evaluating complex RL algorithms with their prebuilt environments; however, they currently do not support ROS or have a straightforward process for transferring learned policies to the real world.

Even though so many frameworks are available for simulating RL environments, scaling to the real world from them is not as easy as the frameworks that use ROS-based architectures. The current ROS-based RL frameworks such as `Openai_ros` package and `gym-gazebo` use the Gazebo simulator and provide prebuilt environments and tools for creating single-task learning environments. However, they currently do not offer options to run multiple parallel environments, which is needed for meta-learning research (Table 1). Therefore, it is impossible to use the current ROS-based frameworks to scale up to evaluate complex RL tasks.

III. THE PROPOSED FRAMEWORK

The framework of the MultiROS is structured based on the typical reinforcement learning architecture. Here we have an agent who selects an *Action* from the pool of available

¹ <https://mujoco.org/>

² <https://pybullet.org/wordpress/>

Actions and executes it in the *Environment* according to the agent's policy. After the environment processes the received action, it output *Observations* and a scalar *Reward* back to the agent. When the agent receives feedback from the environment, it will pick a new action to maximise the total future reward. This process repeats for multiple steps while the agent learns from exploring the environment until it reaches a terminal state (Done). Then we reset the whole environment and repeat the process for multiple episodes to let the agent keep sampling experience and keep improving its policy until it finds the optimal policy to perform the task.

Here the DRL algorithm is the agent, and the proposed framework provides all the functionality of a training environment. MultiROS uses a modular architecture composed of four major components to provide these functions. They are the Gazebo simulator (physics engine), Gazebo-Environment class, Robot-Environment class, and Task-Environment class. The Gazebo-Environment (*GazeboEnv*), which utilises the OpenAI Gym (*gym.Env*), is the class that specifies the connection to the Gazebo simulator. It provides the functionality to reset, pause and resume the simulation. Then the Robot-Environment (*RobotEnv*) class inherits from the *GazeboEnv* and provides the necessary functions to read sensors and move the robot. Finally, the Task-Environment (*TaskEnv*), which inherits from the *RobotEnv*, specifies the structure of the task that the agent needs to learn. Then from the Task-environment, it outputs *Observations*, *Reward* and the *Done* flag (True/False) to the agent. Depending on the requirements, the task can be modelled as a multi-goal environment where it outputs the *Achieved Goal* and the *Desired Goal* together with the *Observations* of the environment. (Figure 1.)

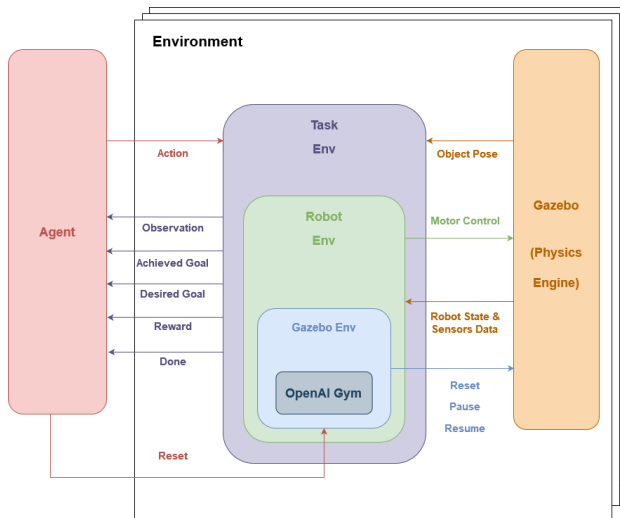


Figure 1. Architecture of the MultiROS Framework

The following sections explain the detailed design of each significant component of the MultiROS architecture.

A. Gazebo Simulator (Physics Engine)

MultiROS uses the Gazebo simulator to simulate all the environments with this framework. Gazebo is an open-source 3D robotics simulator that supports multiple physics engines, rendering, sensors and actuator control packages. It is usually

one of the first simulation software most beginners use in their journey to become ROS programmers. It is also highly compatible with ROS and has extensive community support from roboticists and most hardware manufacturers. Therefore, we can utilise familiar tools like URDF³ files to describe the robot and SDF⁴ files for objects/task scenarios to create simulations. One of the other advantages of the Gazebo simulator is that it supports multiple physics engines. This physics engine is the core of the simulation software that approximates physical system equations, soft-body dynamics and hard-body dynamics, including collision detection. Therefore, having multiple physics engines permits users to apply domain randomisation techniques [21] to effectively sample experiences from various simulated environments with randomised physics properties. This helps the agent find an optimal policy that generalises to work across all the environments. Domain randomisation is one of the approaches that can close the reality gap of sim2real transfer.

One of the essential functions of a reinforcement learning simulation framework is visualising the task. This rendering of the task is helpful in visually inspecting the environment for debugging and finetuning the learning algorithms. However, the rendering of the simulation is not strictly necessary during the training process, and it typically uses a significant amount of computational resources. Therefore, the headless mode of the Gazebo provides the ability to simulate without rendering the environment.

B. Gazebo Environment

The Gazebo-Environment (*GazeboEnv*) is the primary building block of the MultiROS class architecture. It exploits the standardised way of creating a reinforcement learning environment of OpenAI for the MultiROS. This allows the users to apply the familiar structure of OpenAI for initialising (*gym.make*), resetting (*env.reset*), and taking a step (*env.step*) in the environment. The following sections describe the attributes and structure of the *GazeboEnv* class.

Interface with the Gazebo: Since MultiROS is running on the Gazebo simulator, this script communicates with the Gazebo simulator to change the initial physics properties, pause, resume and reset the simulator.

Parallel Simulations: The *GazeboEnv* script offers the ability to run multiple parallel simulations and access data without interference from each other. When initialising an environment using MultiROS (*gym.make*), it will assign different ports for the ROS-Master⁵ and the Gazebo-Master environment variable of each task environment. Then using these ports, MultiROS can access each environment to read and write data simultaneously.

Reproducibility (Seed): One of the most critical aspects of research experiments is that they should be reproducible by others. This is not easy in a reinforcement learning approach where many random elements are present. To address this matter, we implemented a seed function where

³ <http://wiki.ros.org/urdf>

⁴ <http://sdformat.org/>

⁵ <http://wiki.ros.org/rosmaster>

the users can initialise a seed value for all the random value generations in the environment to make their results reproducible. Applying this function, we can also use random seeds to initialise the same tasks in parallel with different physics engines to generalise RL algorithms for continuous control tasks [22].

Apart from the stated functions, the *GazeboEnv* class defines several methods users must override in the *RobotEnv* and the *TaskEnv* classes. Then using those functions, the *GazeboEnv* class defines the main functions to *Reset* and take a *Step* in the training environment.

C. Robot Environment

The Robot-Environment (*RobotEnv*) specifies all the functions of sensors and the robots associated with training the tasks. This class describes the necessary functions for reading the robot-related observations and motor control methods for moving the robots. The modular structure of this *RobotEnv* helps users create any number of Task-Environments using a single robot environment. The following sections describe the composition of the *RobotEnv* class.

Sensors and Actuators: This framework use ROS-services, ROS-actions and ROS-topics to communicate with sensors and actuators. So, the framework utilise ROS's pub/sub (publish/subscribe) structure to read data from all the environment's sensors (Robots and others) and control the robot arm and other actuators. Therefore, the *RobotEnv* is to initialise the Publishers, Subscribers and Services and the call-back functions for getting data from the simulation.

Communication: It is necessary to verify if all the systems (Publishers, Subscribers and Services) are working properly before reading data and sending commands to the simulation. Therefore, this function confirms the proper operation of all the systems used in the task.

D. Task Environment

Task-Environment (*TaskEnv*) class is the main script that specifies all the functionality of the task. This means it defines the main components of the reinforcement learning environment, such as the action space, observation space, setting actions, stopping conditions and reward architecture. Since the *TaskEnv* inherits from the *RobotEnv* class, the methods defined in the *GazeboEnv* class that are specific to the task are overridden here in the *TaskEnv*. These overridden methods are implemented using the helper functions created in the *RobotEnv* to read/write data from/to the Gazebo simulator. Refer to the following for the basic structure of the *TaskEnv* class.

Action Space & Observation Space: Describes the basic structure of the environment. The action space defines the numerical composition of the legitimate actions that the agent can apply to interact with the environment. The observation space defines the structure of what the agent perceives as legitimate values for the state of the environment. Usually, action and observation spaces in RL environments are specified as Discrete or Continuous.

However, in robotic RL environments, the observations usually contain readings from the robot's sensors and other environmental attributes described in the vector form. Therefore, in most robotics-related RL research, the observation space contains continuous values while actions-space values can be continuous or discrete. Since this framework inherits from the OpenAI Gym, the observation space and the action space are represented by the *Box* class for continuous and *Discrete* class for the discrete type of data.

Setting Actions: The *Step* function of the *GazeboEnv* utilise this method when the agent selects an action to interact with the environment. Then, depending on the action space, it executes the appropriate ROS commands using the methods specified in the *RobotEnv* to move the robot in the simulator.

Getting Observations: Provides the current state of the environment for the *Step* and the *Reset* functions of the *GazeboEnv*. This method reads robot-related observations (such as joint angles and robot pose) from the helper functions implemented in the *RobotEnv* and combines them with the task-related observations (such as distance to the goal, object pose, and speed) to describe the implementation of the observation space of the environment.

Terminating Conditions (Is Done): Describes the stopping conditions for the task. Usually, the episode is done when the agent achieves its goal. However, there can be other conditions, such as encountering a state where it is impossible to move the robot (Which means the latest action did not succeed). This method is called in each *Step* to verify if the episode is completed.

Calculate Reward: Defines the instantaneous reward architecture for the task. Since the agent picks actions to maximise the total future reward, it guides the agent to find the optimal behaviour for the robot. Generally, the reward functions are defined as either Sparse reward or Dense reward [23]. Here we can calculate the instantaneous reward using the observations and other user-defined helper functions.

Initial Pose and Initialising Environment Variables: In the typical RL learning process, we reset the environment at the beginning of each episode. This means the robot needs to move back to its initial position and the other task-related variables also need to be reinitialised at each reset. These initial conditions depend on the task and typically differ from one environment to another.

Generate Random Start State: Only necessary for tasks that need a random starting state at the beginning of each episode. The distribution for the random starting state depends on the task agent is trying to learn and the constraints of the environment. For example, it can be a different initial robot pose or a random location to pick the objects (such as in pick-and-place tasks). This method allows the agent to actively explore the state space in each episode to overcome convergence to a suboptimal policy.

Generate Sample Goal: For goal-based environments,

the agent samples experience and attempt to achieve the desired goal. Similar to the random start state, the desired goal of the environment can be initialised with a random distribution that depends on the task and the environmental constraints. Usually, the agent samples a goal on each reset of the environment so that it can effectively explore the state space to find the optimal policy.

IV. HOW TO USE MULTIROS

The MultiROS provide tools to create custom environments and a set of prebuilt goal environments for evaluating DRL algorithms. The following selection contains a summarised version of creating custom environments and using prebuilt environments. For the complete guide, we refer the readers to the project repository⁶.

A. Creating Custom Environments

The initial stage of creating a custom environment is building the simulation of the task in the gazebo simulator. First, users can create a URDF for their own robots or use the files provided by the robot manufacturers to spawn robots in the gazebo simulator. Then using the SDF files, users can create or add objects and design the scenario of the environment. Here, users can specify the physics parameters and physics engine they require in the gazebo *world* file.

GazeboEnv: This GazeboEnv class is common for designing any task and can be directly used without making additional changes. The users only need to inherit from the GazeboEnv class and override the defined mandatory support functions in the other two classes.

RobotEnv: Users need to initialise the sensors and actuators they need to train the task in the RobotEnv. Then users can access the data from the simulation to implement any number of helper functions that may require in the TaskEnv class to take action, define observations, calculate the reward and specify the task's stopping conditions.

TaskEnv: Users need to define the observation space, action space, reward architecture and terminating conditions using the helper functions defined in the TaskEnv and the RobotEnv. These functions shape the structure of the task, and its implementation solely depends on the developer's intuition. Since we are inheriting from the Gym package, users can register different versions of their custom environments using the OpenAI Gym registration method.

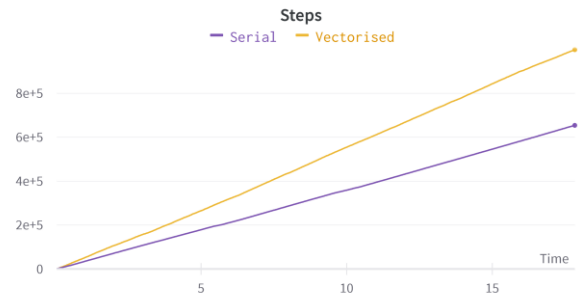
B. Using Prebuilt Environments

MultiROS currently supports three prebuilt task environments for each Fetch robot and ABB IRB 1200 robot, making six prebuilt task environments. The Fetch and the ABB robot have different degree-of-freedom (DOF), link lengths and end-effectors. These two robots also differ in their controller technologies for controlling joint actuators, and they are directly controlled via *ros-controllers*⁷. This

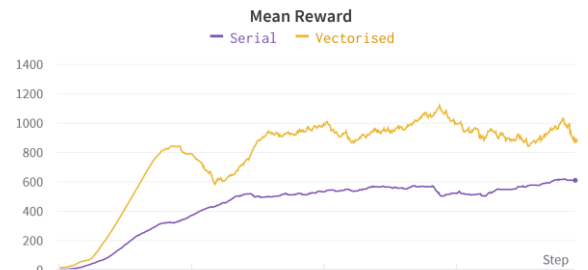
package's current prebuilt tasks are multi-goal environments created to validate goal-oriented continuous control RL algorithms. When initialising the environments, the users can pass arguments to select the reward architecture (Sparse/Dense), rendering and the physics engine. By default, all the predefined tasks use Open Dynamics Engine (ODE) as the physics engine and render the appropriate simulation of the task. The MultiROS package also includes all the necessary simulation files to directly use the MultiROS for testing and validating DRL algorithms. Please refer to the GitHub page for more information on the available task environments.

V. FRAMEWORK EVALUATION

To validate our work, we investigated one of the applications of MultiROS framework. Here we used the MultiROS to vectorise the same environment to see how collecting experience from multiple parallel environment instances will affect the agent's learning process. Theoretically, it should enable the agent to effectively sample experience from multiple instances to accelerate the DRL training procedure. One of the bottlenecks of performing robotic RL tasks with ROS lies in the wait times for the Gazebo simulator. Since sampling experience is much slower with the Gazebo simulator than with lightweight simulators like MuJoCo, we can use the MultiROS framework to vectorise the environment by initialising multiple simulator instances in parallel. For the experiment, we used the Proximal Policy Optimization (PPO) [24] algorithm with the "FetchReachROS-v1" environment of the MultiROS. This environment uses the Fetch robot to reach a random goal location (a point in 3D space) within the workspace of the robot.



(a) Collected samples during the training



(b) Mean cumulative reward during the training

Figure 2. Quantitative results for serial and vectorised training

⁶ <https://github.com/ncbdrck/MultiROS>

⁷ http://wiki.ros.org/ros_control

Here, we ran the experiment with four vectorised environments and compared the results with a serialised environment. We plotted the mean total reward and the number of samples collected for both approaches to understand the effect of vectorisation on the learning process. Figure 2 shows that the vectorised approach collects more samples and outputs higher mean cumulative rewards during the training than the serialised method. Therefore, MultiROS reduces the time cost of the Deep RL research by providing functionality to sample from multiple Gazebo instances to accelerate the task's learning. This example is only one of the applications of MultiROS, and users can use the current prebuilt environments or create their own to train/ evaluate meta-learning and multi-task learning RL algorithms.

VI. CONCLUSION & FUTURE WORK

This work presents MultiROS, an open-source framework to design, test and validate robotic environments for reinforcement learning applications. The main objective of this work is to introduce features lacking in the current ROS-based RL frameworks for meta-learning, multi-task and transfer learning research. MultiROS's ongoing and future activities will introduce additional robots and vision sensors to extend task environments for meta-learning and multi-agent RL research. Another future direction is to enhance MultiROS to support real robots, evaluate the sim-to-real transfer from this framework, and switch between real and simulated robots for sampling experience. Lastly, we want to explore the possibility of adding support for next-generation Ignition-Gazebo⁸ and other simulators since the Open-Source Robotics Foundation (OSRF) will end support for Gazebo 11 in early 2025.

ACKNOWLEDGEMENT

This publication has emanated from research conducted with the financial support of Science Foundation Ireland (SFI) under Grant Number SFI 16/RC/3918, co-funded by the European Regional Development Fund.

REFERENCES

- [1] J. Kober, J. A. Bagnell, and J. Peters, 'Reinforcement learning in robotics: A survey', *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, Sep. 2013, doi: 10.1177/0278364913495721.
- [2] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, 'A Brief Survey of Deep Reinforcement Learning', *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017, doi: 10.1109/MSP.2017.2743240.
- [3] G. Dulac-Arnold, D. Mankowitz, and T. Hester, 'Challenges of Real-World Reinforcement Learning', *ArXiv190412901 Cs Stat*, Apr. 2019.
- [4] F. Muratore, M. Gienger, and J. Peters, 'Assessing Transferability From Simulation to Reality for Reinforcement Learning', *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 4, pp. 1172–1183, Apr. 2021, doi: 10.1109/TPAMI.2019.2952353.
- [5] J. Collins, D. Howard, and J. Leitner, 'Quantifying the Reality Gap in Robotic Manipulation Tasks', in *2019 International Conference on Robotics and Automation (ICRA)*, May 2019, pp. 6706–6712, doi: 10.1109/ICRA.2019.8793591.
- [6] M. Quigley *et al.*, 'ROS: an open-source Robot Operating System', in *ICRA workshop on open source software*, 2009, vol. 3, no. 3.2, p. 5.
- [7] N. Koenig and A. Howard, 'Design and use paradigms for Gazebo, an open-source multi-robot simulator', in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No. 04CH37566)*, Sep. 2004, vol. 3, pp. 2149–2154 vol.3, doi: 10.1109/IROS.2004.1389727.
- [8] A. S. Polydoros and L. Nalpantidis, 'Survey of Model-Based Reinforcement Learning: Applications on Robotics', *J. Intell. Robot. Syst.*, vol. 86, no. 2, pp. 153–173, May 2017, doi: 10.1007/s10846-017-0468-y.
- [9] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, 'How to train your robot with deep reinforcement learning: lessons we have learned', *Int. J. Robot. Res.*, vol. 40, no. 4–5, pp. 698–721, Apr. 2021, doi: 10.1177/0278364920987859.
- [10] Alberto Ezquerro, Miguel Angel Rodriguez and Ricardo Tellez (of The Construct), 'openai_ros - ROS Wiki', 2019, http://wiki.ros.org/openai_ros (accessed May 06, 2021).
- [11] I. Zamora, N. G. Lopez, V. M. Vilches, and A. H. Cordero, 'Extending the OpenAI Gym for robotics: a toolkit for reinforcement learning using ROS and Gazebo', *ArXiv160805742 Cs*, Feb. 2017.
- [12] G. Brockman *et al.*, 'OpenAI Gym', *ArXiv160601540 Cs*, Jun. 2016.
- [13] Y. Tassa *et al.*, 'dm_control: Software and Tasks for Continuous Control', *Softw. Impacts*, vol. 6, p. 100022, Nov. 2020, doi: 10.1016/j.simpa.2020.100022.
- [14] Q. Gallouédec, N. Cazin, E. Dellandréa, and L. Chen, 'panda-gym: Open-source goal-conditioned environments for robotic learning', *ArXiv210613687 Cs*, Dec. 2021.
- [15] P. Henderson, W.-D. Chang, F. Shkurti, J. Hansen, D. Meger, and G. Dudek, 'Benchmark Environments for Multitask Learning in Continuous Domains', *ArXiv170804352 Cs*, Aug. 2017.
- [16] M. Vavrecka, N. Sokovnin, M. Mejdrachova, and G. Sejnova, 'MyGym: Modular Toolkit for Visuomotor Robotic Tasks', in *2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI)*, Nov. 2021, pp. 279–283, doi: 10.1109/ICTAI52525.2021.00046.
- [17] Y. Zhu, J. Wong, A. Mandekar, and R. Martín-Martín, 'robosuite: A Modular Simulation Framework and Benchmark for Robot Learning', *ArXiv200912293 Cs*, Sep. 2020.
- [18] L. Fan *et al.*, 'SURREAL: Open-Source Reinforcement Learning Framework and Robot Manipulation Benchmark', in *Proceedings of The 2nd Conference on Robot Learning*, Oct. 2018, pp. 767–782.
- [19] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, 'RLBench: The Robot Learning Benchmark amp; Learning Environment', *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3019–3026, Apr. 2020, doi: 10.1109/LRA.2020.2974707.
- [20] T. Yu *et al.*, 'Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning', in *Proceedings of the Conference on Robot Learning*, May 2020, pp. 1094–1100.
- [21] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, 'Domain randomization for transferring deep neural networks from simulation to the real world', in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sep. 2017, pp. 23–30, doi: 10.1109/IROS.2017.8202133.
- [22] A. Parvez Mohammed and M. Valdenegro-Toro, 'Can Reinforcement Learning for Continuous Control Generalize Across Physics Engines?', Oct. 2020.
- [23] A. Mohtasib, G. Neumann, and H. Cuayáhuil, 'A Study on Dense and Sparse (Visual) Rewards in Robot Policy Learning', in *Towards Autonomous Robotic Systems*, Cham, 2021, pp. 3–13, doi: 10.1007/978-3-030-89177-0_1.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, 'Proximal Policy Optimization Algorithms', *ArXiv170706347 Cs*, Aug. 2017.

⁸ <https://ignitionrobotics.org/home>